



Huffman Coding

Basic Idea



Note:

Fixed-length encoding
ASCII, Unicode

Variable-length encoding : assign longer code words to less frequent characters, shorter code words to more frequent characters.

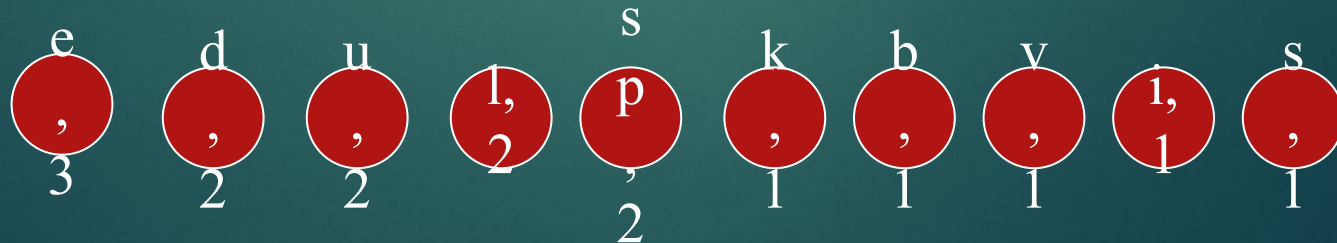
Huffman Coding



- Huffman codes can be used to compress information
 - ▶ Like WinZip – although WinZip doesn't use the Huffman algorithm
 - ▶ JPEGs do use Huffman as part of their compression process

Huffman Coding (Cont.)

- As an example, lets take the string:
“duke blue devils”
- We first do a frequency count of the characters:
 - ▶ e:3, d:2, u:2, l:2, space:2, k:1, b:1, v:1, i:1, s:1
- Next we use a Greedy algorithm to build up a Huffman Tree
 - ▶ We start with nodes for each character

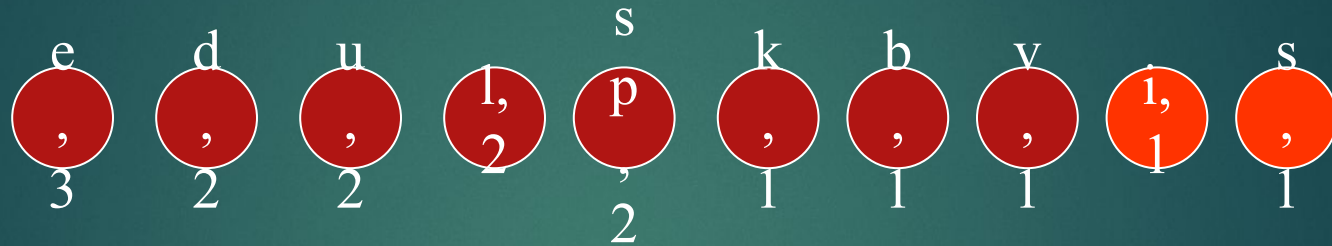


Huffman Coding (Cont.)

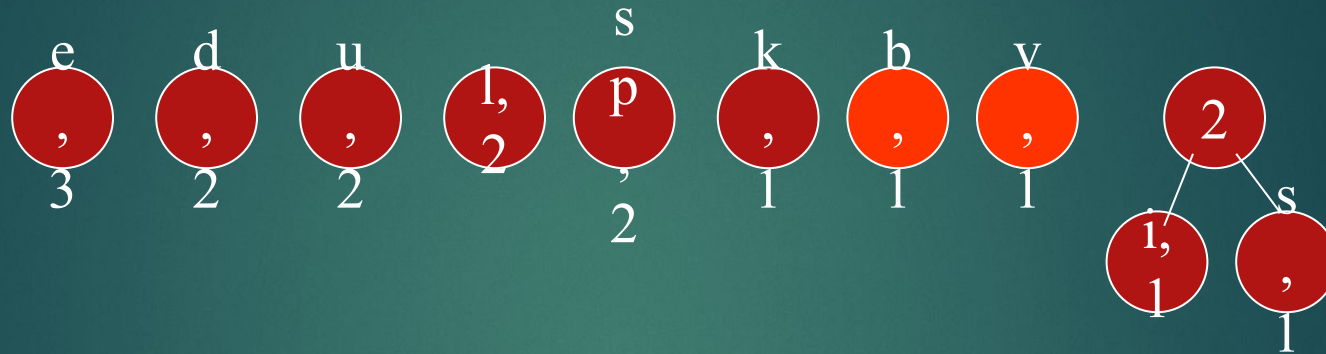


- We then pick the nodes with the smallest frequency and combine them together to form a new node.
 - ▶ The selection of these nodes is the Greedy part
- The two selected nodes are removed from the set, but replace by the combined node.
- This continues until we have only 1 node left in the set.

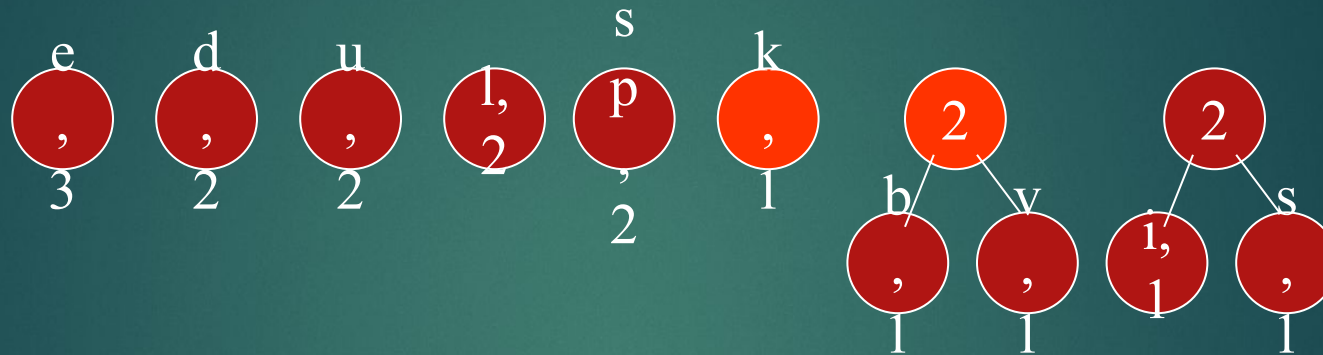
Huffman Coding (Cont.)



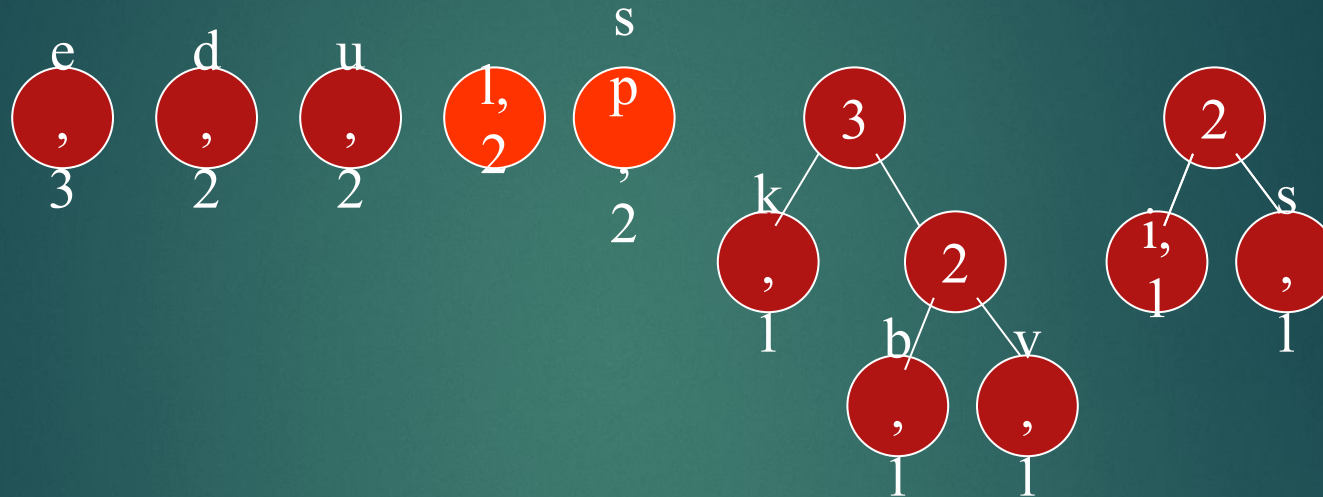
Huffman Coding (Cont.)



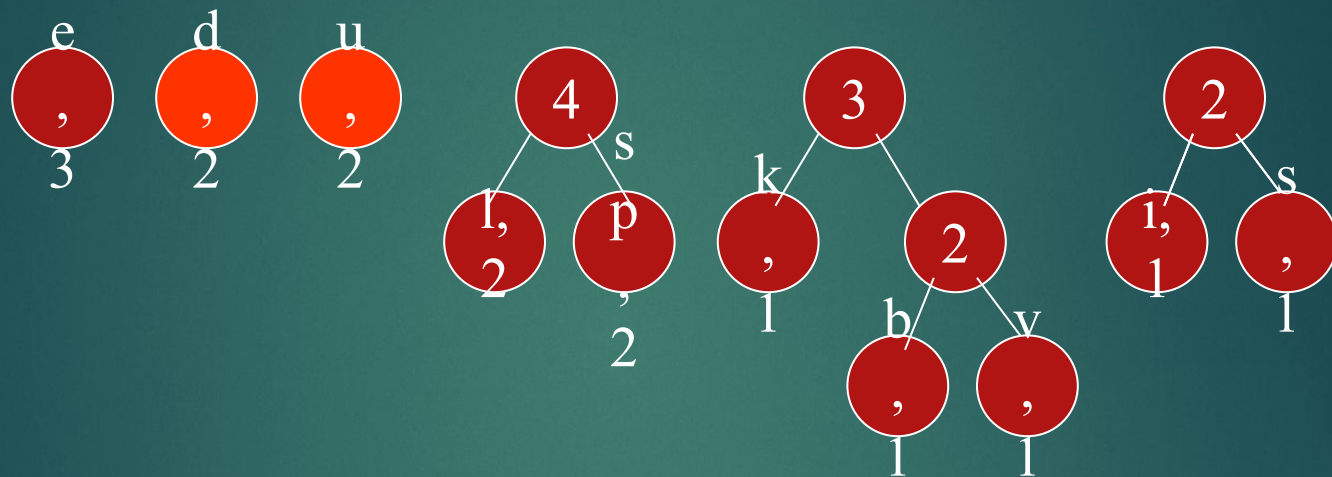
Huffman Coding (Cont.)



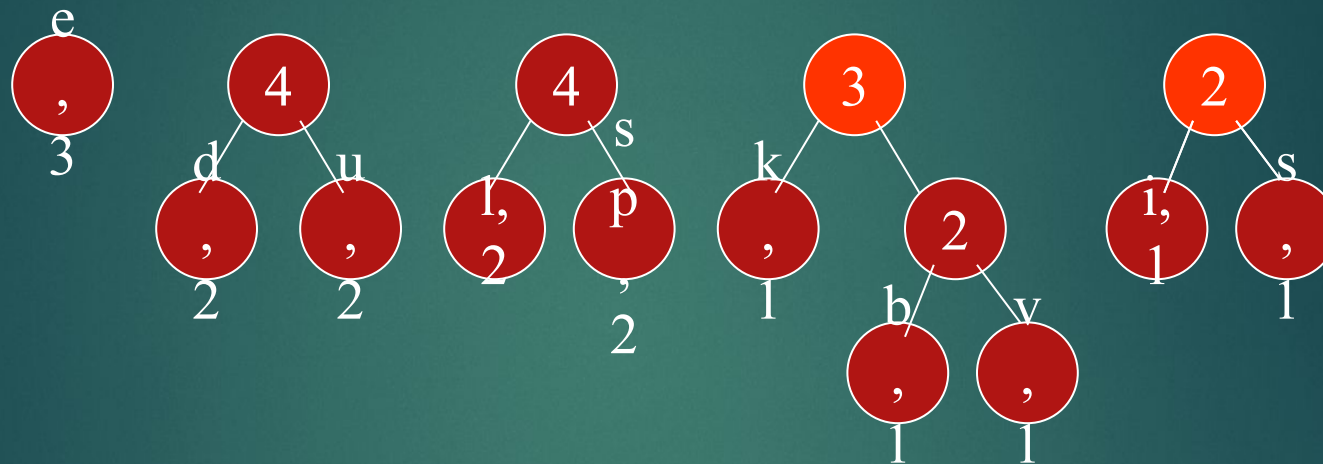
Huffman Coding (Cont.)



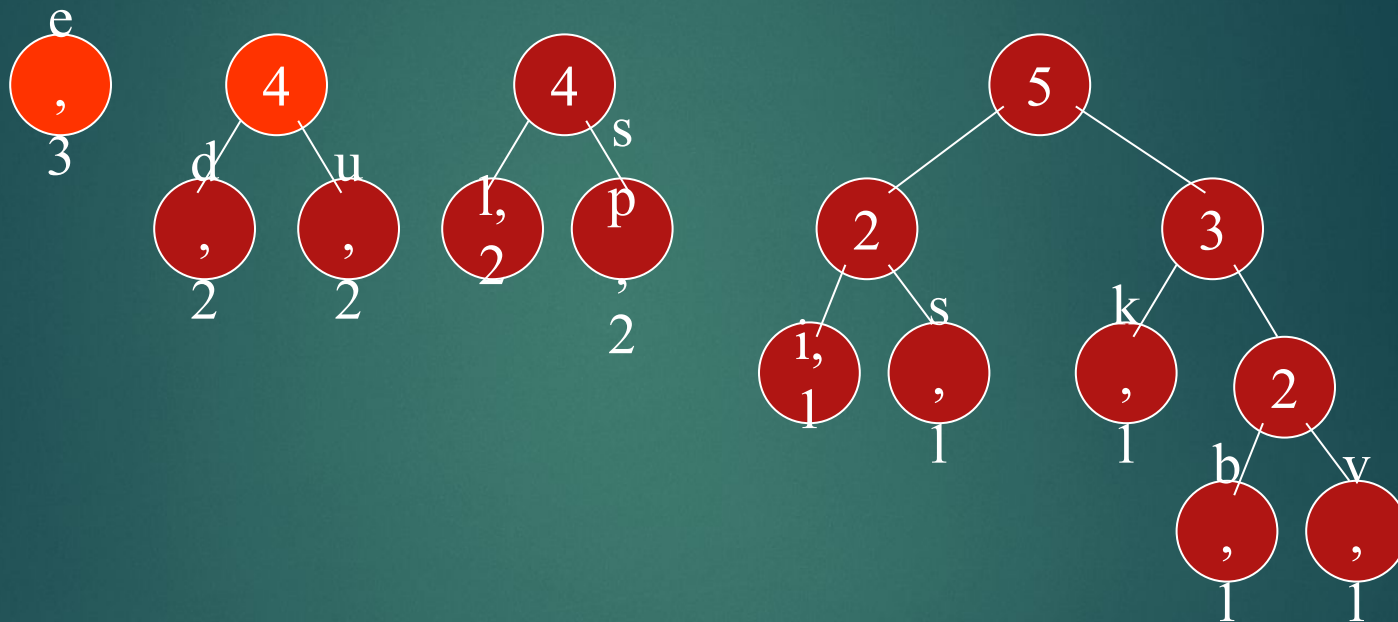
Huffman Coding (Cont.)



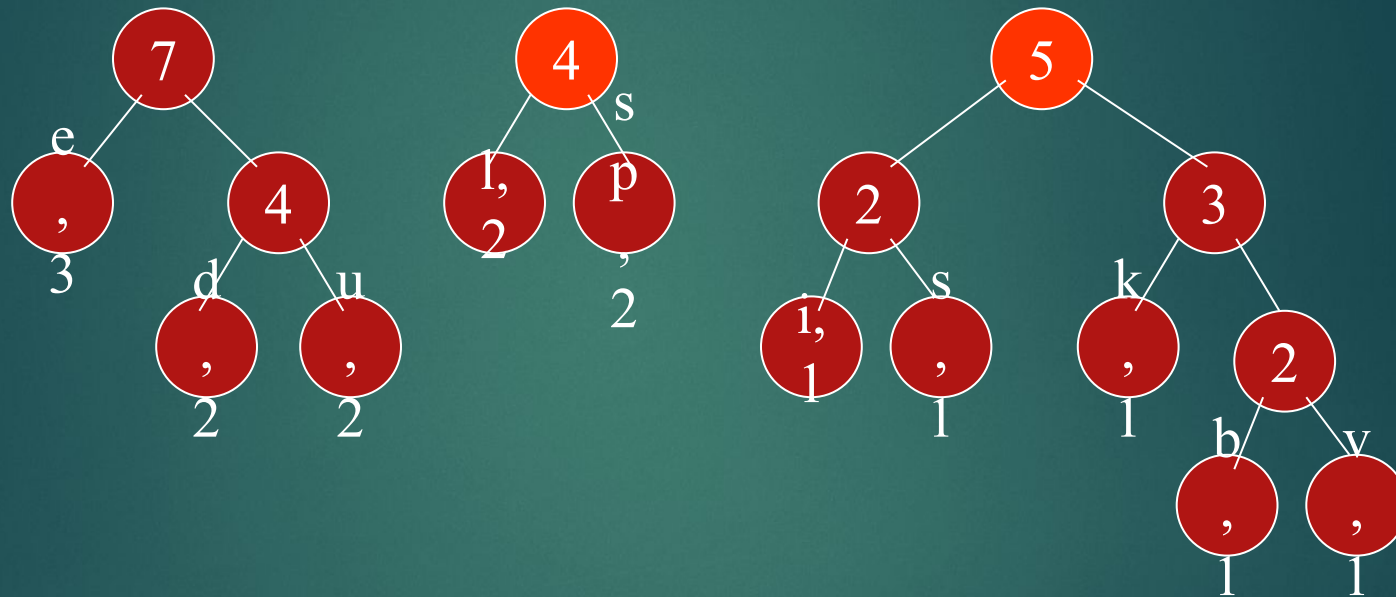
Huffman Coding (Cont.)



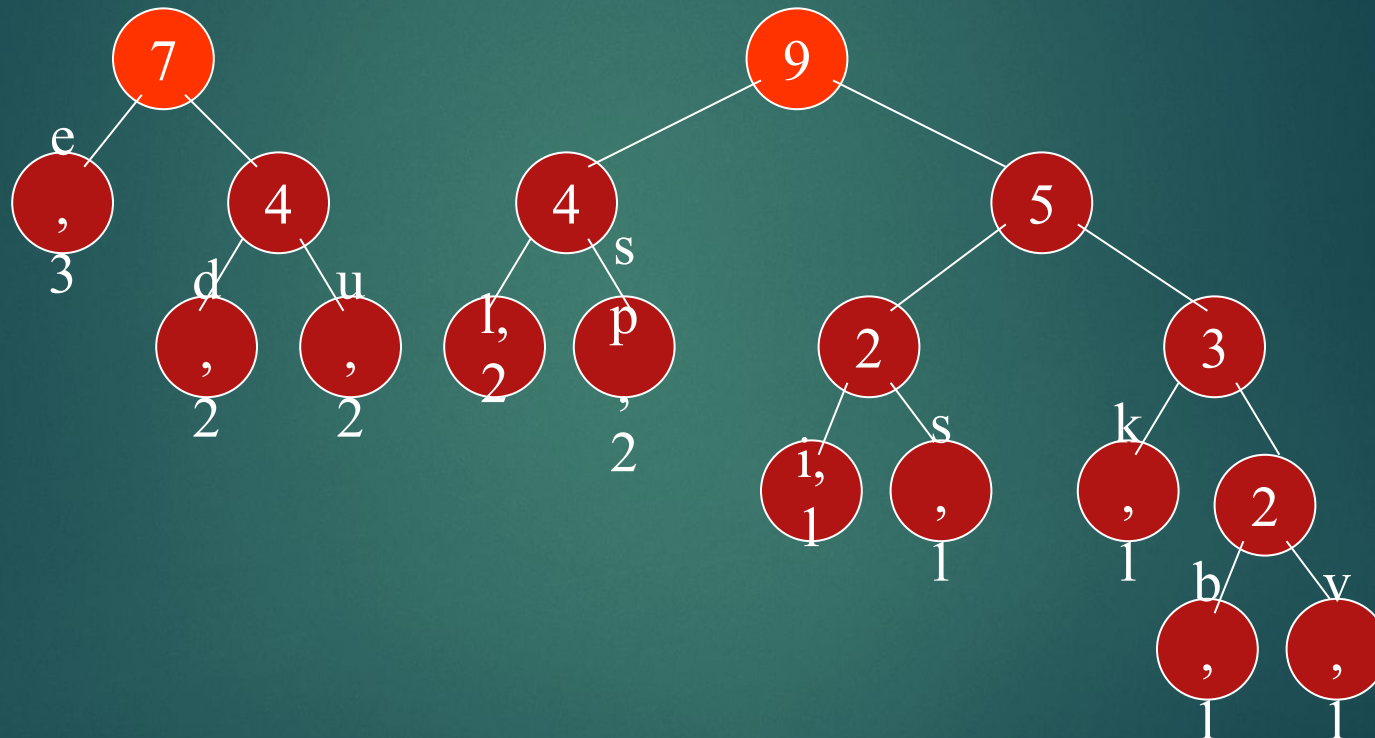
Huffman Coding (Cont.)



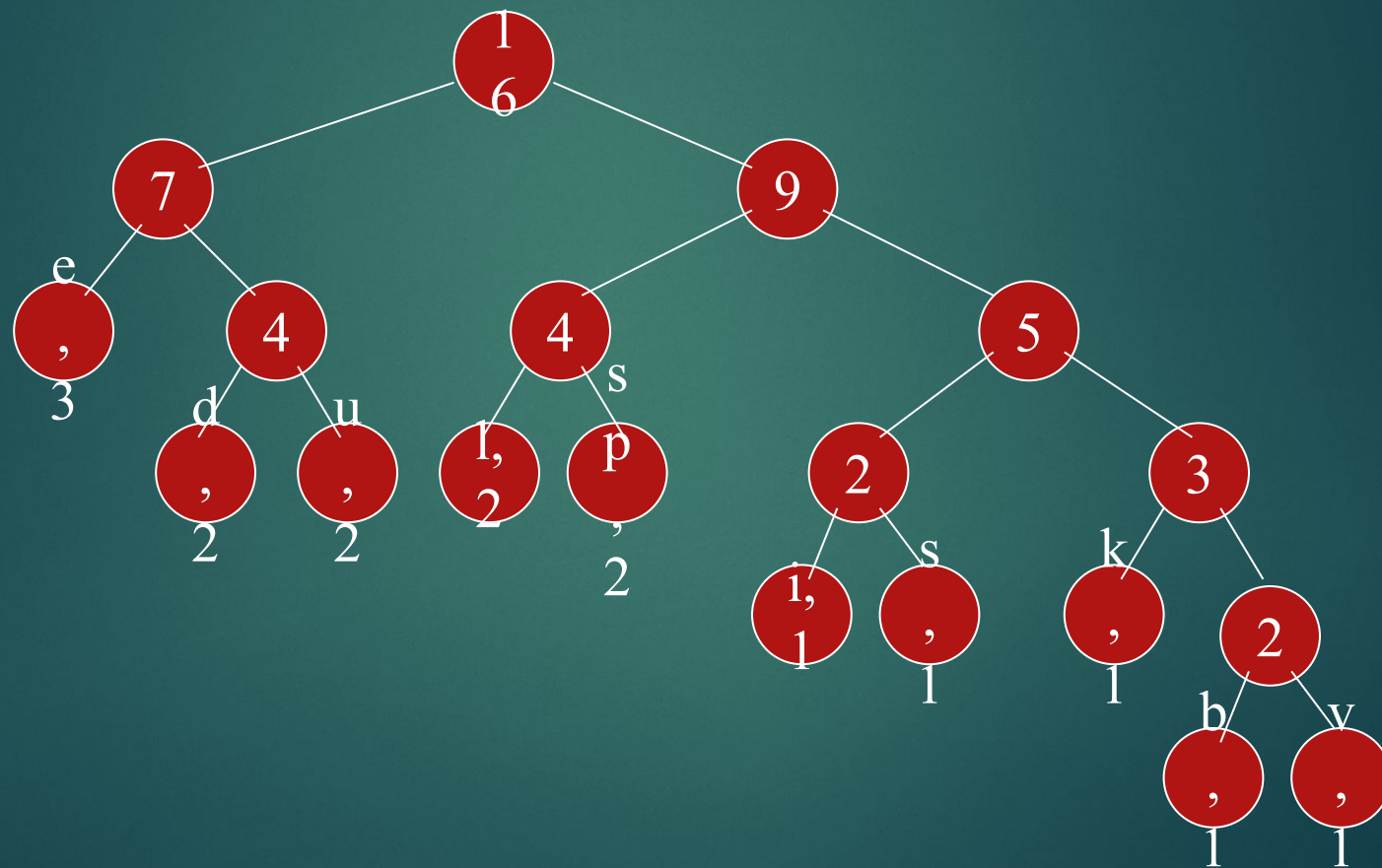
Huffman Coding (Cont.)



Huffman Coding (Cont.)



Huffman Coding (Cont.)

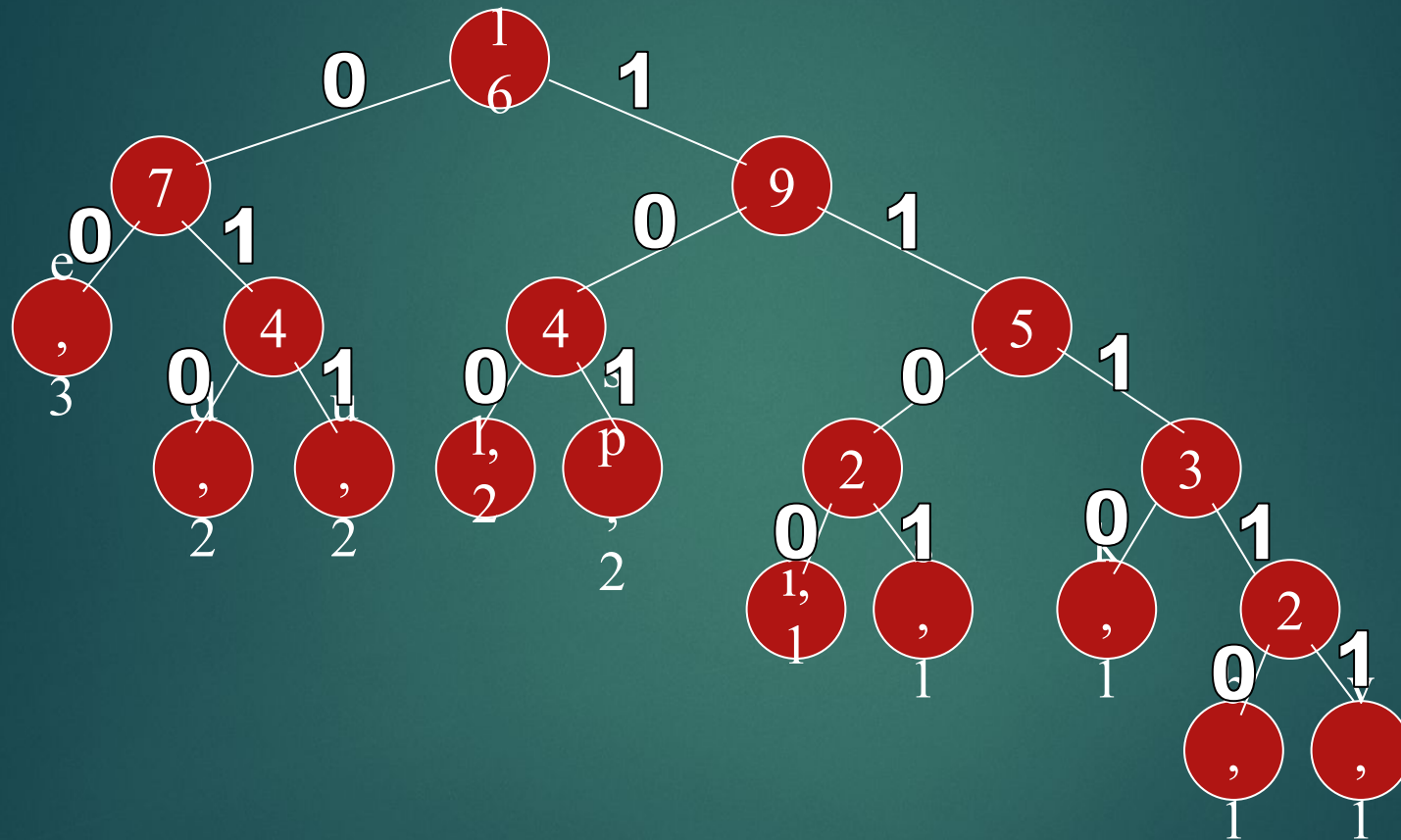


Huffman Coding (Cont.)



- Now we assign codes to the tree by placing a 0 on every left branch and a 1 on every right branch.
- A traversal of the tree from root to leaf give the Huffman code for that particular leaf character.
- Note that no code is the prefix of another code.

Huffman Coding (Cont.)



e	00
d	010
u	011
l	100
sp	101
i	1100
s	1101
k	1110
b	11110
v	11111

Huffman Coding (Cont.)

- These codes are then used to encode the string.

- Thus, “duke blue devils” turns into:

010 011 1110 00 101 11110 100 011 00 101 010 00 11111 1100 100 1101

- When grouped into 8-bit bytes:

01001111 10001011 11101000 11001010 10001111 11100100 1101xxxx

- Thus it takes 7 bytes of space compared to 16 characters * 1 byte/char = 16 bytes uncompressed

Huffman Coding

- Uncompressing works by reading in the file bit by bit.
 - ▶ Start at the root of the tree
 - ▶ If a 0 is read, head left
 - ▶ If a 1 is read, head right
 - ▶ When a leaf is reached decode that character and start over again at the root of the tree
- Thus, we need to save Huffman table information as a header in the compressed file.
 - ▶ Doesn't add a significant amount of size to the file for large files (which are the ones you want to compress anyway)
 - ▶ Or we could use a fixed universal set of codes/frequencies

Most important properties of Huffman Coding

- Unique Prefix Property: No Huffman code is a prefix of any other Huffman code
 - ▶ For example, 101 and 1010 cannot be Huffman codes. Why?
- Optimality: The Huffman code is a minimum-redundancy code (given an accurate data model)
 - ▶ The two least frequent symbols will have the same length for their Huffman code, whereas symbols occurring more frequently will have shorter Huffman codes
 - ▶ It has been shown that the average code length of an information source S is strictly less than $\eta + 1$, i.e.

$$\eta \leq l' < \eta + 1$$

END